STM32基础 - FLASH编程

课程目标

1. 熟悉FLASH编程

课程试验

1. 利用STM32完成FLASH读写，掉电不丢失

课堂引入

如何防止产品被偷?
1.禁用SWD引脚，只能解决使用stlink读flash的问题，解决不了使用串口bootloader读flash的问题
2.开启读保护
3.开启侵入监测

STM32F1系列的FLASH包含程序存储器、系统存储器和选项字节三个部分，通过闪存存储器接口（外设）可以对程序存储器和选项字节进行擦除和编程

读写FLASH的用途:

> - 利用程序存储器的剩余空间来保存掉电不丢失的用户数据
> - 通过在程序中编程（IAP），实现程序的自我更新

授课进程

一、FLASH简介

STM32F1系列的FLASH包含程序存储器、系统存储器和选项字节三个部分，通过闪存存储器接口（外设）可以对程序存储器和选项字节进行擦除和编程

读写FLASH的用途:

- 利用程序存储器的剩余空间来保存掉电不丢失的用户数据
- 通过在程序中编程（IAP），实现程序的自我更新

在线编程（In-Circuit Programming – ICP）用于更新程序存储器的全部内容，它通过JTAG、SWD协议或系统加载程序（Bootloader）下载程序

在程序中编程（In-Application Programming – IAP）可以使用微控制器支持的任一种通信接口下载程序

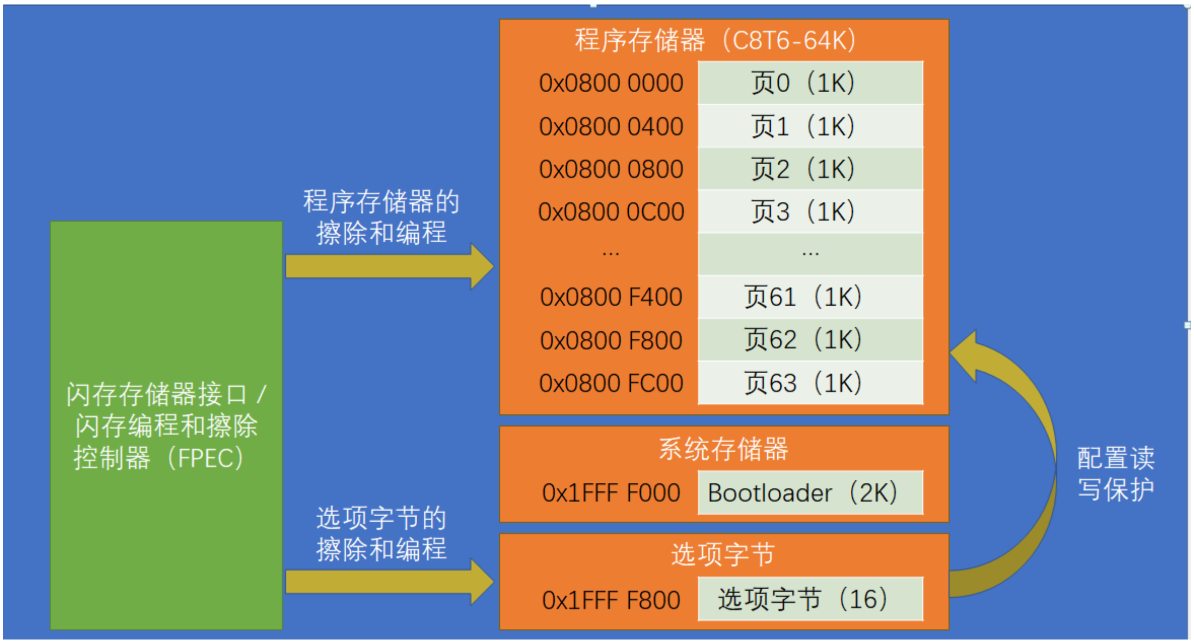
二、闪存

表2　闪存模块组织(中容量产品)

块	名称	地址范围	长度(字节)
主存储器	页0	0x0800 0000 – 0x0800 03FF	1K
	页1	0x0800 0400 – 0x0800 07FF	1K
	页2	0x0800 0800 – 0x0800 0BFF	1K
	页3	0x0800 0C00 – 0x0800 0FFF	1K
	页4	0x0800 1000 – 0x0800 13FF	1K
	. . .	. . .	. . .
	页127	0x0801 FC00 – 0x0801 FFFF	1K
信息块	启动程序代码	0x1FFF F000 – 0x1FFF F7FF	2K
	用户选择字节	0x1FFF F800 – 0x1FFF F80F	16
闪存存储器接口寄存器	FLASH_ACR	0x4002 2000 – 0x4002 2003	4
	FLASH_KEYR	0x4002 2004 – 0x4002 2007	4
	FLASH_OPTKEYR	0x4002 2008 – 0x4002 200B	4
	FLASH_SR	0x4002 200C – 0x4002 200F	4
	FLASH_CR	0x4002 2010 – 0x4002 2013	4
	FLASH_AR	0x4002 2014 – 0x4002 2017	4
	保留	0x4002 2018 – 0x4002 201B	4
	FLASH_OBR	0x4002 201C – 0x4002 201F	4
	FLASH_WRPR	0x4002 2020 – 0x4002 2023	4

三、FLASH操作

1、基本结构


程序存储器（C8T6-64K）
0x0800 0000
页0（1K）
0x0800 0400
页1（1K）
0x0800 0800
页2（1K）
0x0800 0C00
页3（1K）
…
…
0x0800 F400
页61（1K）
0x0800 F800
页62（1K）
0x0800 FC00
页63（1K）
系统存储器
0x1FFF F000
Bootloader（2K）
选项字节
0x1FFF F800
选项字节（16）
闪存存储器接口 / 闪存编程和擦除控制器（FPEC）
程序存储器的擦除和编程
选项字节的擦除和编程
配置读写保护

2、FLASH解锁

•FPEC共有三个键值：

> RDPRT键 = 0x000000A5
>
> KEY1 = 0x45670123
>
> KEY2 = 0xCDEF89AB

•解锁：

> 复位后，FPEC被保护，不能写入FLASH_CR
>
> 在FLASH_KEYR先写入KEY1，再写入KEY2，解锁
>
> 错误的操作序列会在下次复位前锁死FPEC和FLASH_CR

•加锁：

> 设置FLASH_CR中的LOCK位锁住FPEC和FLASH_CR

3、使用指针访问存储器

•使用指针读指定地址下的存储器：

```
uint16_t Data = *((__IO uint16_t *)(0x08000000));
```

•使用指针写指定地址下的存储器：

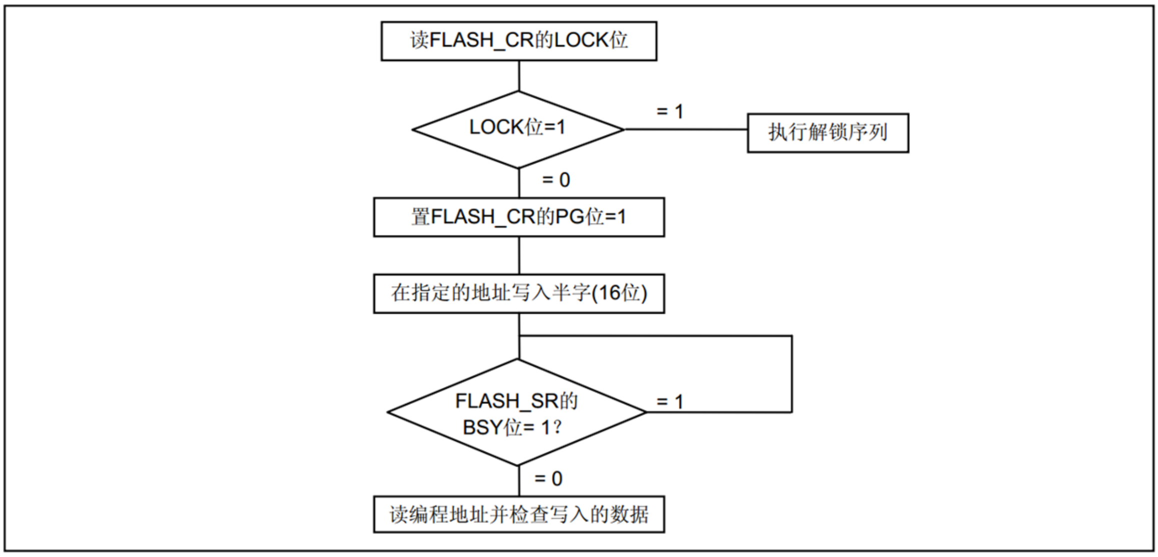
```
*((__IO uint16_t *)(0x08000000)) = 0x1234;
```

•其中：

```
#define  __IO  volatile
```

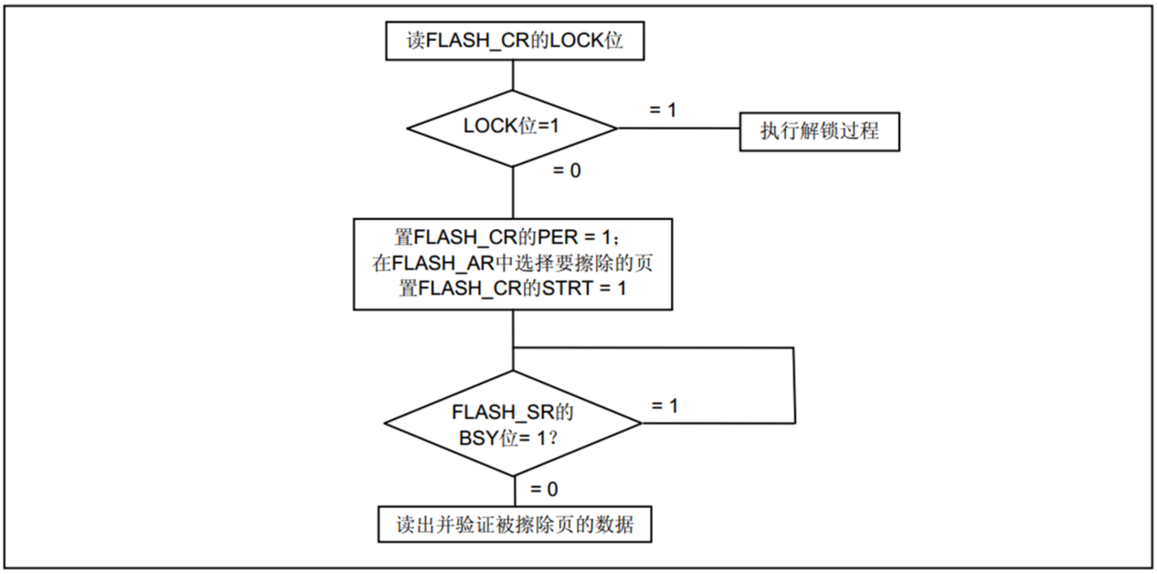
4、程序存储器编程

图一　编程过程


读FLASH_CR的LOCK位
LOCK位=1
= 1
执行解锁序列
= 0
置FLASH_CR的PG位=1
在指定的地址写入半字(16位)
FLASH_SR的
BSY位= 1?
= 1
= 0
读编程地址并检查写入的数据

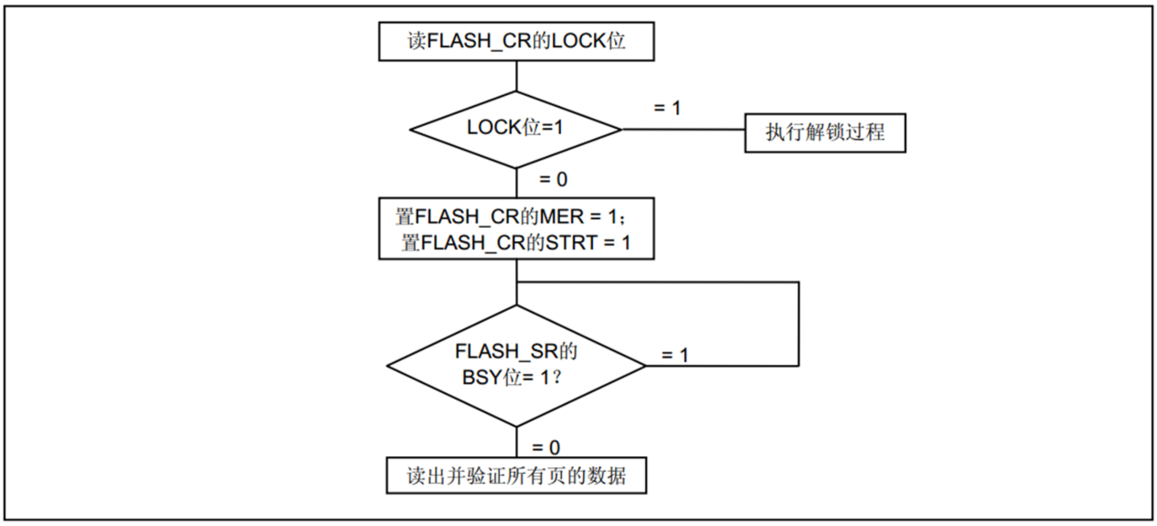
5、程序存储器页擦除

图二　闪存页擦除过程


读FLASH_CR的LOCK位
LOCK位=1
= 1
执行解锁过程
= 0
置FLASH_CR的PER = 1；
在FLASH_AR中选择要擦除的页
置FLASH_CR的STRT = 1
FLASH_SR的
BSY位= 1?
= 1
= 0
读出并验证被擦除页的数据

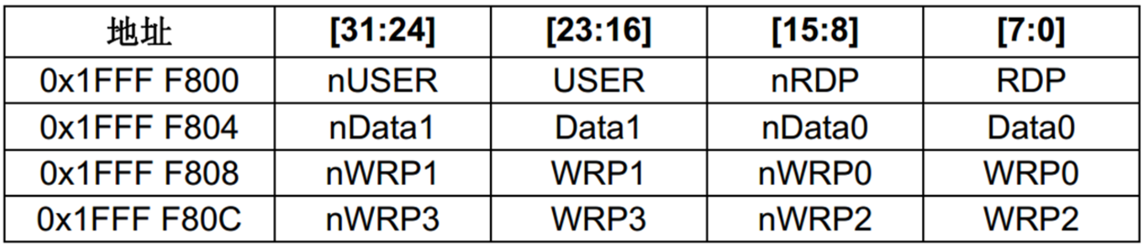
6、程序存储器全擦除

图三　闪存全擦除过程


读FLASH_CR的LOCK位
LOCK位=1
= 1
执行解锁过程
= 0
置FLASH_CR的MER = 1；
置FLASH_CR的STRT = 1
FLASH_SR的
BSY位= 1?
= 1
= 0
读出并验证所有页的数据

7、选项字节

表6 信息块的组织结构



地址	[31:24]	[23:16]	[15:8]	[7:0]
0x1FFF F800	nUSER	USER	nRDP	RDP
0x1FFF F804	nData1	Data1	nData0	Data0
0x1FFF F808	nWRP1	WRP1	nWRP0	WRP0
0x1FFF F80C	nWRP3	WRP3	nWRP2	WRP2

> RDP：写入RDPRT键（0x000000A5）后解除读保护
>
> USER：配置硬件看门狗和进入停机/待机模式是否产生复位
>
> Data0/1：用户可自定义使用
>
> WRP0/1/2/3：配置写保护，每一个位对应保护4个存储页（中容量）

8、选项字节编程

> 检查FLASH_SR的BSY位，以确认没有其他正在进行的编程操作
>
> 解锁FLASH_CR的OPTWRE位
>
> 设置FLASH_CR的OPTPG位为1
>
> 写入要编程的半字到指定的地址
>
> 等待BSY位变为0
>
> 读出写入的地址并验证数据

9、选项字节擦除

> 检查FLASH_SR的BSY位，以确认没有其他正在进行的闪存操作
>
> 解锁FLASH_CR的OPTWRE位
>
> 设置FLASH_CR的OPTER位为1
>
> 设置FLASH_CR的STRT位为1
>
> 等待BSY位变为0
>
> 读出被擦除的选择字节并做验证

10、器件电子签名

电子签名存放在闪存存储器模块的系统存储区域，包含的芯片识别信息在出厂时编写，不可更改，使用指针读指定地址下的存储器可获取电子签名

闪存容量寄存器：

> 基地址：0x1FFF F7E0
>
> 大小：16位

产品唯一身份标识寄存器：

> 基地址： 0x1FFF F7E8
>
> 大小：96位

四、FLASH操作实验

1、实验目的

读写stm32内部的flash闪存

2、程序设计

MyFlash.c

```c
#include "stm32f10x.h"                  // Device header

/*
字(Word)：32位长的数据或指令
半字(Half Word)：16位长的数据或指令
字节(Byte)：8位长的数据或指令
*/

uint32_t MyFLASH_ReadWord(uint32_t Address) // 地址都是32位
{
    // #define     __IO    volatile                  /*!< defines 'read / write' permissions   */
    //volatile  提醒编译器不进行优化，比如放入缓存，把无用重复代码删除。
    return *((__IO uint32_t *)(Address));   //使用指针访问指定地址下的数据并返回
}

//FLASH读取一个16位的半字
uint16_t MyFLASH_ReadHalfWord(uint32_t Address)
{
    return *((__IO uint16_t *)(Address));   //使用指针访问指定地址下的数据并返回
}

//FLASH读取一个8位的字节
uint8_t MyFLASH_ReadByte(uint32_t Address)
{
    return *((__IO uint8_t *)(Address));    //使用指针访问指定地址下的数据并返回
}

//全部擦除
void MyFLASH_EraseAllPages(void)
{
    FLASH_Unlock();                 //解锁
    FLASH_EraseAllPages();          //全擦除
    FLASH_Lock();                   //加锁
}
//FLASH页擦除
void MyFLASH_ErasePage(uint32_t PageAddress)
```

```
{
    FLASH_Unlock();                 //解锁
    FLASH_ErasePage(PageAddress);   //页擦除
    FLASH_Lock();                   //加锁
}
//Address 要写入数据的字地址
void MyFLASH_ProgramWord(uint32_t Address, uint32_t Data)
{
    FLASH_Unlock();                         //解锁
    FLASH_ProgramWord(Address, Data);       //编程字
    FLASH_Lock();                           //加锁
}

//Address 要写入数据的半字地址
void MyFLASH_ProgramHalfWord(uint32_t Address, uint16_t Data)
{
    FLASH_Unlock();                         //解锁
    FLASH_ProgramHalfWord(Address, Data);   //编程半字
    FLASH_Lock();                           //加锁
}

void MyFLASH_ReadProtection(FunctionalState NewState) //读保护
{
    FLASH_Unlock();                         //解锁
    FLASH_ReadOutProtection(NewState);      //读保护
    FLASH_Lock();                           //加锁
}
```

main.c

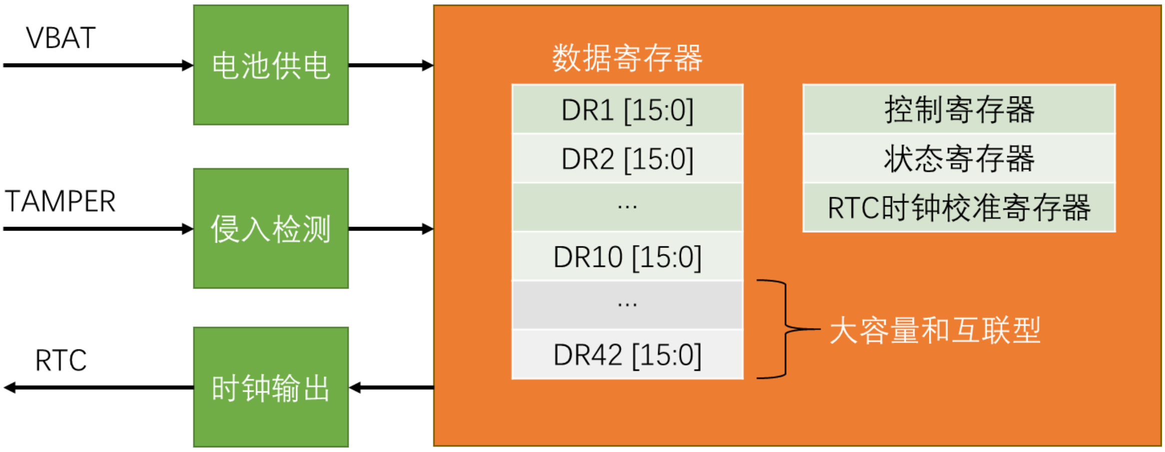
```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "MyFlash.h"

int main(void)
{
    OLED_Init();

    MyFLASH_ReadProtection(DISABLE); // 开启后不能写入了

    OLED_Clear();
    uint8_t data = MyFLASH_ReadByte(0x08000000);

    OLED_ShowHexNum(1,1,data,8);
    OLED_ShowString(2,1,"Monday");

    while(1)
    {
    }
}
```

五、BKP

1、基本介绍

BKP（Backup Registers）备份寄存器或者叫后备寄存器。可用于存储用户应用程序数据。BKP的特性是当VDD电源被切断，它们仍然由VBAT维持供电。当系统在待机模式下被唤醒，或系统复位或电源复位时，它们也不会被复位。这里VDD就是系统的主电源，供电电压是2.0-3.6V，VBAT就是备用电池电源，供电电压是1.8-3.6V。TAMPER引脚产生的侵入事件将所有备份寄存器内容清除，可以用于设备防拆检测。RTC引脚输出RTC校准时钟、RTC闹钟脉冲或者秒脉冲，可以用于为其它设备提供时钟。存储RTC时钟校准寄存器用于校准时钟。可以看出BKP和RTC的关联程度很高，设计者就是把RTC的这两个功能放到BKP中了。BKP的用户数据存储容量是 20字节（中容量和小容量）/ 84字节（大容量和互联型），对于STM32F103C8T6芯片来说，其BKP容量是20字节。

2、基本结构



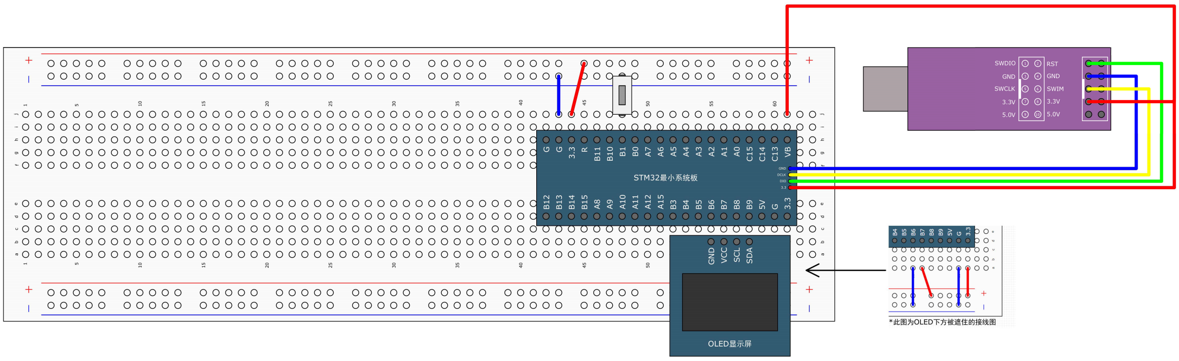
BKP基本结构图。图中橙色部分可以称为后备区域，BKP处于后备区域，但后备区域不只有BKP，还有RTC的相关电路也位于后备区域。STM32后备区域的特性就是当VDD主电源掉电时，后备区域仍然可以由VBAT的备用电池供电。当VDD主电源上电时，后备区域供电会由VBAT切换到VDD，也就是主电源有电时，VBAT是不会用的，这样可以节省电池电量。然后BKP是位于后备区域的，BKP里主要有数据寄存器、控制寄存器、状态寄存器和RTC时钟校准寄存器等。其中数据寄存器是主要部分，它用来存储数据的。每个数据寄存器都是16位的，也就是一个数据寄存器可以存2个字节。对于中容量和小容量的设备，里面有DR1、DR2一直到DR10，总共10个数据寄存器。一个进入寄存器存2个字节，所以总容量是20个字节。对于大容量和互联型设备，里面除了DR1到DR10，还有DR11、DR12一直到DR42，总共42个数据寄存器，因此其总容量是84个字节。BKB还有另外几个功能。侵入检测，可以从PC13位置的TEMPER引脚，引入一个检测信号，当TEMPER产生上升沿或者下降沿时，清除BKP所有的内容，以保证安全。时钟输出，可以把RTC的相关时钟，从PC13位置的RTC引脚输出数据，供外部使用。其中输出校准时钟时再配合BKP中的时钟校准寄存器，可以对RTC的误差进校准。

六、读写BKP实验

1、实验目的

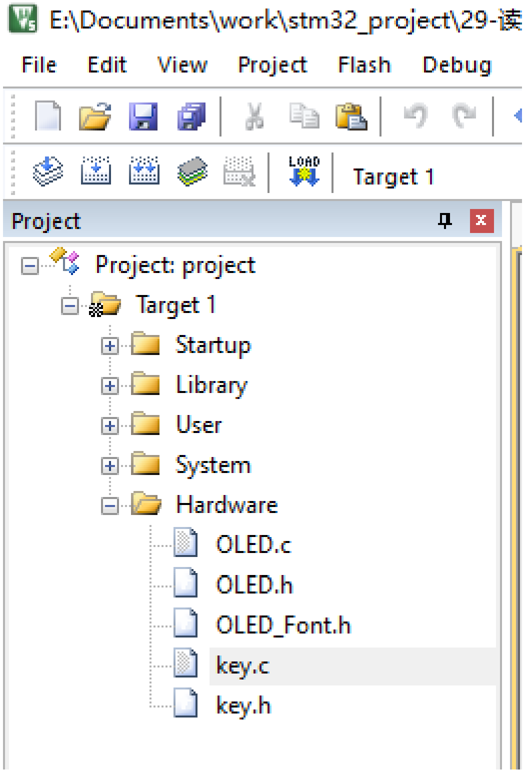
通过控制STM32的备份寄存器的读写，实现主电源掉电，写入备份寄存器的数据依然能够正确读取，并将过程数据显示在OLED上。

2、硬件电路接线图


STM32最小系统板
GND
VCC
SCL
SDA
OLED显示屏
SWDIO
GND
SWCLK
3.3V
5.0V
RST
GND
SWIM
3.3V
5.0V
B4
B5
B6
B7
B8
B9
5V
G
3.3
*此图为OLED下方被遮住的接线图

3、程序设计

工程结构如下图：


E:\Documents\work\stm32_project\29-读
File
Edit
View
Project
Flash
Debug
LOAD
Target 1
Project
Project: project
Target 1
Startup
Library
User
System
Hardware
OLED.c
OLED.h
OLED_Font.h
key.c
key.h

其中Hardware组与之前工程相比，没有增加单独的驱动，只是放入了之前课程用过的按键驱动，主要代码都在main.c中。具体代码展示如下：

key.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"

void KEY_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1 | GPIO_Pin_11;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;

    GPIO_Init(GPIOB, &GPIO_InitStructure);
}
```

```
uint8_t KEY_GetNum(void) {
    uint8_t KeyNum = 0;
    if (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_1) == 0) {
        Delay_ms(20); // 按键后防抖
        while (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_1) == 0); // 等待按键抬起
        Delay_ms(20); // 按键抬起后防抖
        KeyNum = 1;
    }
    if (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_11) == 0) {
        Delay_ms(20);
        while (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_11) == 0);
        Delay_ms(20);
        KeyNum = 2;
    }
    return KeyNum;
}
```

key.h

```
# ifndef __KEY_H
# define __KEY_H

void KEY_Init(void);
uint8_t KEY_GetNum(void);

# endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "key.h"

uint8_t KeyNum;          // 定义用于接收按键键码的变量
uint16_t ArrayWrite[] = {0x1234, 0x5678};   // 定义要写入数据的测试数组
uint16_t ArrayRead[2];                      // 定义要读取数据的测试数组

int main(void){

    OLED_Init();
    KEY_Init();

    OLED_ShowString(1, 1, "W:");
    OLED_ShowString(2, 1, "R:");

    RCC_APB1PeriphClockCmd(RCC_APB1Periph_PWR, ENABLE);     // 打开PWR时钟
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_BKP, ENABLE);     // 打开BKP时钟

    // 备份寄存器访问使能
    PWR_BackupAccessCmd(ENABLE);

    while(1){
        KeyNum = KEY_GetNum();
        if (KeyNum == 1) {
            ArrayWrite[0]++;
            ArrayWrite[1]++;

            BKP_WriteBackupRegister(BKP_DR1, ArrayWrite[0]);    // 写入测试数据到备份寄存器DR1
            BKP_WriteBackupRegister(BKP_DR2, ArrayWrite[1]);    // 写入测试数据到备份寄存器DR2

            OLED_ShowHexNum(1, 3, ArrayWrite[0], 4);
            OLED_ShowHexNum(1, 8, ArrayWrite[1], 4);
        }

        ArrayRead[0] = BKP_ReadBackupRegister(BKP_DR1);     // 读取备份寄存器DR1的数据
        ArrayRead[1] = BKP_ReadBackupRegister(BKP_DR2);     // 读取备份寄存器DR2的数据

        OLED_ShowHexNum(2, 3, ArrayRead[0], 4);
        OLED_ShowHexNum(2, 8, ArrayRead[1], 4);
    }
}
```

代码完成后，编译并下载到硬件中执行，会发现实验结果是主电源掉电，写入备用寄存器的数据读取没有问题，OLED显示正确，符合预期设计。

课程小结

1、理解FLASH操作的步骤和方法。

2、理解什么是BKP，供电方式，操作步骤。

随堂作业

1、对比FLASK和BKP的操作区别。